

3300 project 1

Question 1: Use `wget` on `example.com` to view the last modified date of the webpage. What was the HTTP return status given and what command was used to do this? (The command should not download the file! Hint: Look into the `wget` man page.)

```
karinajadia@Ubuntu-2004:~$ wget --spider -S example.com
Spider mode enabled. Check if remote file exists.
--2025-02-20 15:49:05-- http://example.com/
Resolving example.com (example.com)... 23.192.228.84, 23.192.228.80, 96.7.128.1
98, ...
Connecting to example.com (example.com)[23.192.228.84]:80... connected.
HTTP request sent, awaiting response...
HTTP/1.1 200 OK
Content-Type: text/html
ETag: "84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT
Cache-Control: max-age=2977
Date: Thu, 20 Feb 2025 20:49:05 GMT
Connection: keep-alive
Vary: Accept-Encoding
Length: unspecified [text/html]
Remote file exists and could contain further links,
but recursion is disabled -- not retrieving.
```

I used `wget --spider -S example.com` to view the last modified date. The return status was 200, which means it was successful.

Question 2: Run `tracert -I google.com`, show your screenshot and illustrate the meaning of this command.

```
karinajadia@Ubuntu-2004:~$ tracert -I google.com
tracert to google.com (142.250.80.110), 64 hops max
 1  142.250.80.110  10.177ms  11.014ms  10.207ms
```

Tracert shows the number of routers that data packets pass through to reach `google.com`, so in this case it shows that data packets go through 64 or less routers from my computer to `google.com`.

Question 3: In your own words describe what a DNS resource record (RR) is. Now using the command line tool `nslookup` find the MX resource record of `uconn.edu`. What does this resource record mean?

A DNS resource record contains data about domain names and IP addresses and other information. It is used in a DNS database file to make internet traffic more efficient.

```
karinajadia@Ubuntu-2004:~$ nslookup -type=MX uconn.edu
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
uconn.edu    mail exchanger = 10 uconn-edu.mail.protection.outlook.com.
```

This resource record shows the IP address and the mail server associated with `uconn.edu`.

Question 4: What does the command `nslookup -type=ns .` do? Explain its output. (Note: the `.` is part of the command!)

It lists the authoritative root name servers in the DNS system. All of the name servers in the output are root name servers that handle directing DNS queries to the correct TLD name servers.

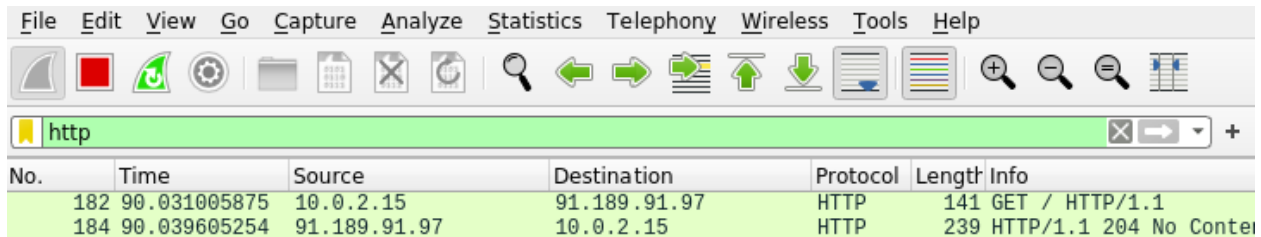
```
karinajadia@Ubuntu-2004:~$ nslookup -type=ns .
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
.            nameserver = f.root-servers.net.
.            nameserver = j.root-servers.net.
.            nameserver = m.root-servers.net.
.            nameserver = a.root-servers.net.
.            nameserver = g.root-servers.net.
.            nameserver = l.root-servers.net.
.            nameserver = h.root-servers.net.
.            nameserver = e.root-servers.net.
.            nameserver = i.root-servers.net.
.            nameserver = d.root-servers.net.
.            nameserver = k.root-servers.net.
.            nameserver = b.root-servers.net.
.            nameserver = c.root-servers.net.
```

Part 1: HTTP

Question 1: Open Chromium and navigate to <http://httpbin.org>. Find the HTTP packet that corresponds to the initial request that your computer made. Take a screenshot of this packet. What HTTP method did your computer use to make this request?

Question 2: Find the HTTP packet that corresponds to the initial response the server made to your request. Take a screenshot of this packet. What HTTP status code did the server return? What is the content type of the response the server is sending back?



The screenshot shows the Wireshark interface with the 'http' filter applied. The packet list shows two packets:

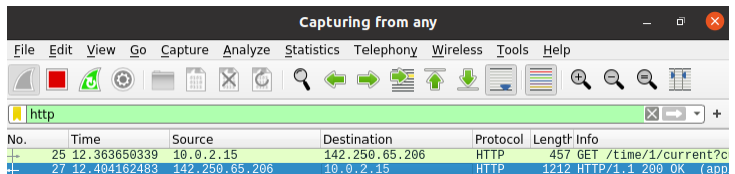
No.	Time	Source	Destination	Protocol	Length	Info
182	90.031005875	10.0.2.15	91.189.91.97	HTTP	141	GET / HTTP/1.1
184	90.039605254	91.189.91.97	10.0.2.15	HTTP	239	HTTP/1.1 204 No Content

I'm answering question one and two in the screenshot. The first row is the packet that corresponds to the initial request. My computer used a GET method to make the request.

The second row shows the response. The difference between them is the HTTP method used and the source and destination IP addresses. My computer sent a GET request to the server at 91.189.91.97 and that server sent a response (no content was sent back).

Question 3: Use Chromium and navigate to <http://uconn.edu>. Find the HTTP packets that correspond to the initial request and response that your computer made. Take a screenshot of these packets. What's different? Explain.

The difference was the IP address that my computer sent its GET request to, and the response code (200 instead of 204), which means the server sent stuff back to my computer.



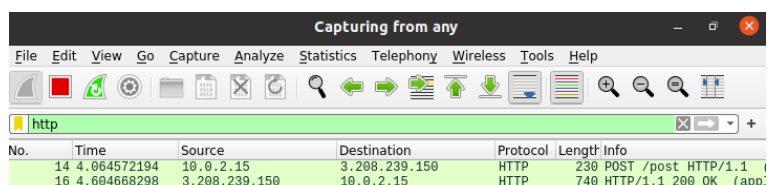
The screenshot shows the Wireshark interface with the 'http' filter applied. The packet list shows two packets:

No.	Time	Source	Destination	Protocol	Length	Info
25	12.363650339	10.0.2.15	142.250.65.206	HTTP	457	GET /time/1/current?c
27	12.404102483	142.250.65.206	10.0.2.15	HTTP	1212	HTTP/1.1 200 OK (app

Question 4: Using Chromium (or any other Linux utility you are comfortable with), find a way to make a HTTP packet with a method other than GET. Take a screenshot of your packet, and explain what you did to create it.

I did a post request to <http://httpbin.org/post>, here's my command: `curl -X POST -d "name=Karina&message=Hello" http://httpbin.org/post`. The post request means I sent something to httpbin.org instead of getting something from the website.

```
karina@karina-ubuntu-2004:~$ curl -X POST -d "name=Karina&message=Hello" http://httpbin.org/post
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "message": "Hello",
    "name": "Karina"
  },
  "headers": {
    "Accept": "*/*",
    "Content-Length": "25",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "curl/7.68.0",
    "X-Amzn-Trace-Id": "Root=1-67b7e5ae-1f08c80c74452c2147fd4c49"
  },
  "json": null,
  "origin": "137.99.143.59",
  "url": "http://httpbin.org/post"
}
```



The screenshot shows the Wireshark interface with the 'http' filter applied. The packet list shows two packets:

No.	Time	Source	Destination	Protocol	Length	Info
14	4.064572194	10.0.2.15	3.208.239.150	HTTP	230	POST /post HTTP/1.1
16	4.604668298	3.208.239.150	10.0.2.15	HTTP	740	HTTP/1.1 200 OK (app

Part 2: DNS

5. Open Chromium and navigate to www.example.com. Were any steps taken by your computer before the web page was loaded? If so, using your captured packets in Wireshark, find the packets that allow your computer to successfully load <http://www.example.com>. Take a screenshot of these packets and explain why you think these are the correct packets. What's the IP address of www.example.com?

Source	Destination	Protocol	Length	Info	
10.0.2.15	10.0.2.3	DNS	88	Standard query 0xbab7 AAAA www.example.com	
10.0.2.15	10.0.2.3	DNS	88	Standard query 0x1722 A www.example.com OP	
10.0.2.15	10.0.2.3	DNS	88	Standard query 0xace8 Unknown (65) www.exa	
Source	Destination	Protocol	Length	Info	
10.0.2.15	185.125.190.97	TCP	76	35486 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1	
10.0.2.15	185.125.190.49	TCP	76	52338 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1	
185.125.190.49	10.0.2.15	TCP	62	80 → 52338 [SYN, ACK] Seq=0 Ack=1 Win=65535	
185.125.190.97	10.0.2.15	TCP	62	80 → 35486 [SYN, ACK] Seq=0 Ack=1 Win=65535	
Time	Source	Destination	Protocol	Length	Info
57 5.432917974	10.0.2.15	185.125.190.97	HTTP	143	GET / HTTP/1.1
59 5.506013071	185.125.190.97	10.0.2.15	HTTP	241	HTTP/1.1 204 No Content

These are the packets that allowed my computer to load [example.com](http://www.example.com). I think these are the correct packets because they follow the correct flow that would be made when loading a website. First, my computer made a DNS query for the IP address of [example.com](http://www.example.com) and got a result, then it established a TCP connection to connect to [example.com](http://www.example.com)'s server, finally it made an HTTP get response to [example.com](http://www.example.com)'s IP address and got a response. The IP address of [example.com](http://www.example.com) is 185.125.190.97, which is where it made the TCP and HTTP requests.

6. Open a terminal window. Flush your DNS cache. Using `wget`, download the same content of www.example.com with its IP address you discovered in question 5, without sending DNS requests. What command did you use to accomplish that? Take a screenshot of related packets and explain why you think these are the correct packets.

Time	Source	Destination	Protocol	Length	Info
18.308018772	10.0.2.15	185.125.190.97	TCP	76	35670 → 80 [SYN] Seq=0 Win=6
18.313089039	fe80::2	ff02::1	ICMPv6	112	Router Advertisement from 52
18.335566006	fe80::799c:d05c:143...	ff02::16	ICMPv6	152	Multicast Listener Report Me
18.381691967	185.125.190.97	10.0.2.15	TCP	62	80 → 35670 [SYN, ACK] Seq=0
18.381760407	10.0.2.15	185.125.190.97	TCP	56	35670 → 80 [ACK] Seq=1 Ack=1
18.382439455	10.0.2.15	185.125.190.97	HTTP	197	GET / HTTP/1.1
18.383436563	185.125.190.97	10.0.2.15	TCP	62	80 → 35670 [ACK] Seq=1 Ack=1
18.456124170	185.125.190.97	10.0.2.15	HTTP	222	HTTP/1.1 204 No Content
18.456166720	10.0.2.15	185.125.190.97	TCP	56	35670 → 80 [ACK] Seq=142 Ack:
18.457251807	10.0.2.15	185.125.190.97	TCP	56	35670 → 80 [FIN, ACK] Seq=14:
18.457934532	185.125.190.97	10.0.2.15	TCP	62	80 → 35670 [ACK] Seq=167 Ack:
18.463101760	fe80::799c:d05c:143...	ff02::16	ICMPv6	152	Multicast Listener Report Me
18.529684744	185.125.190.97	10.0.2.15	TCP	62	80 → 35670 [FIN, ACK] Seq=16:
18.529752791	10.0.2.15	185.125.190.97	TCP	56	35670 → 80 [ACK] Seq=143 Ack:

```
karina@kali:~$ wget http://185.125.190.97 --no-check-certificate
--2025-02-21 22:13:49-- http://185.125.190.97/
Connecting to 185.125.190.97:80... connected.
HTTP request sent, awaiting response... 204 No Content
2025-02-21 22:13:49 (0.00 B/s) - 'index.html' saved [0]
```

I used `wget http://[example.com IP address]` to bypass the DNS request and get the packets.

I think these are the right packets because these are the TCP and HTTP requests to the correct IP address, so it's following the same steps to get the same result, minus the DNS lookup.

7. Open a terminal window. Using nslookup, find the A records for www.google.com. Take a screenshot of the packets corresponding to your request, and the response from the server. If the request was resolved, what is the IP address you were given for www.google.com?

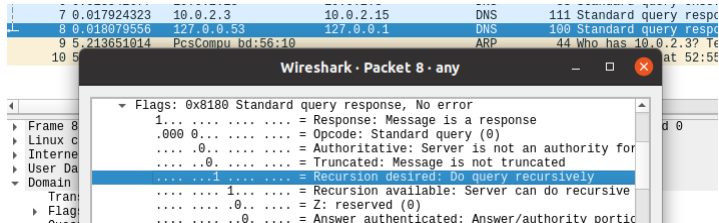
```
karinajadia@Ubuntu-2004:~$ nslookup google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.80.110
Name:   google.com
Address: 2607:f8b0:4006:80d::200e
```

I was given 142.250.80.110 as the IP address for Google. These are the requests that my computer made, all DNS to the IP address for Google.

Time	Source	Destination	Protocol	Length	Info
0.000000000	127.0.0.1	127.0.0.53	DNS	72	Standard query 0xba22 A google
0.000176939	10.0.2.15	10.0.2.3	DNS	83	Standard query 0xca18 A google
0.012599420	10.0.2.3	10.0.2.15	DNS	99	Standard query response 0xca18
0.012853013	127.0.0.53	127.0.0.1	DNS	88	Standard query response 0xba22
0.013436604	127.0.0.1	127.0.0.53	DNS	72	Standard query 0xeb02 AAAA goo
0.013542677	10.0.2.15	10.0.2.3	DNS	83	Standard query 0x3c7b AAAA goo
0.017924323	10.0.2.3	10.0.2.15	DNS	111	Standard query response 0x3c7b
0.018070556	127.0.0.53	127.0.0.1	DNS	100	Standard query response 0xeb02

8. Did your computer want to complete the request recursively? How do you know? Take a screenshot proving your answer.



Yes, it wanted to do it recursively. I checked the flags on the DNS query packet and the flags for recursion (highlighted) and recursion availability are 1, which means it did recursion.

9. Using nslookup, find the A records for uconn.edu. Take a screenshot of the packets corresponding to your request, and the response from the server. If the request was resolved, what is the IP address you were given for uconn.edu?

```
karinajadia@Ubuntu-2004:~$ nslookup uconn.edu
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   uconn.edu
Address: 137.99.146.50
```

The IP address I was given for UConn was 137.99.146.50, and the data packets corresponding to my request.

Time	Source	Destination	Protocol	Length	Info
1 0.000000000	127.0.0.1	127.0.0.53	DNS	71	Standard query 0xf6c6 A uconn
2 0.000172298	127.0.0.53	127.0.0.1	DNS	87	Standard query response 0xf6c6
3 0.000687776	127.0.0.1	127.0.0.53	DNS	71	Standard query 0xba66 AAAA u
4 0.000771986	10.0.2.15	10.0.2.3	DNS	82	Standard query 0x41e8 AAAA u
5 0.065252428	10.0.2.3	10.0.2.15	DNS	145	Standard query response 0x41e8
6 0.065555278	127.0.0.53	127.0.0.1	DNS	71	Standard query response 0xba66

10. What is the authoritative name server for the uconn.edu domain? How do you know? Take a screenshot proving your answer.

The authoritative name server is uconn.edu, which is according to the screenshot I have.

```
karinajadia@Ubuntu-2004:~$ nslookup -type=ns uconn.edu
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
uconn.edu    nameserver = gdc-dr.net.uconn.edu.
uconn.edu    nameserver = msb-ext.net.uconn.edu.
uconn.edu    nameserver = hbl-ext.net.uconn.edu.

Authoritative answers can be found from:
```